

Fundamentos de Inteligência Artificial

Programa de Pós-graduação em Computação

Mestrado em Computação – CDTec/Universidade Federal de Pelotas (UFPel)

TRABALHO DE BUSCAS - SOLUÇÃO DO QUEBRA-CABEÇAS DESLIZANTE

Prof. Ulisses Brisolara Corrêa

Grupo: Andriel, Isabel Salaberry

Objetivo:

Este trabalho tem como objetivo implementar e comparar diferentes algoritmos de busca aplicados ao problema do 8-puzzle. Foram analisados algoritmos de busca não informada e busca informada, avaliando seu desempenho em termos de tempo de execução, quantidade de nós explorados e qualidade da solução.

O problema do Estado Inicial:

Na proposta do quebra-cabeças deslizante apresentado, é importante frisar que nem todo estado inicial possui solução. Isso ocorre devido à propriedade matemática da paridade das inversões.

Uma inversão acontece quando um número maior precede um número menor na sequência do tabuleiro (ignorando o espaço vazio). Para um tabuleiro 3x3 (8-puzzle), um estado só é alcançável a partir do estado objetivo se o número total de inversões for par. Se a configuração inicial resultar em um número ímpar de inversões, o algoritmo entrará em um loop infinito ou esgotará o espaço de busca sem nunca encontrar o objetivo, pois é fisicamente impossível reorganizar as peças apenas deslizando-as.

Considerando que o objetivo do presente trabalho é a implementação e avaliação de diferentes algoritmos de inteligência artificial e não testar todas as possibilidades possíveis de um quebra-cabeças deslizante, usamos como teste no código apenas estados iniciais válidos, como o exemplo abaixo.

```
tabuleiro = (1, 2, 3, 5, 0, 6, 4, 7, 8)
```

1. Busca em Largura (BFS)

A BFS explora todos os nós de um nível antes de avançar para o próximo. Utiliza a estrutura fila (FIFO) e com isso garante uma solução ótima mas com alto consumo de memória, sendo ideal quando se busca o menor número de movimentos.

```
def bfs(initial_state):  
    start_time = time.time()  
  
    start_node = Node(initial_state)
```

```

if start_node.state == GOAL_STATE:
    return start_node, 0, time.time() - start_time

frontier = deque([start_node])
explored = {initial_state}
nodes_expanded = 0

while frontier:
    current_node = frontier.popleft()
    nodes_expanded += 1
    for child in get_successors(current_node):
        if child.state not in explored:
            if child.state == GOAL_STATE:
                return child, nodes_expanded, time.time() -
start_time
            explored.add(child.state)
            frontier.append(child)

    return None, nodes_expanded, time.time() - start_time

```

2.1 Busca em Profundidade (DFS)

A DFS explora o espaço de estados aprofundando-se ao máximo em um caminho antes de retroceder. Utiliza estrutura de pilha (LIFO) com consumo baixo uso de memória e não garante solução ótima, pois pode encontrar soluções rapidamente, mas nem sempre as melhores.

```

def dfs(initial_state, max_nodes=10000):
    start_time = time.time()

    start_node = Node(initial_state)

    stack = [start_node]

```

```

visited = set()

nodes_expanded = 0

while stack:

    current_node = stack.pop()

    nodes_expanded += 1

    # Proteção contra travamento
    if nodes_expanded > max_nodes:

        print("Limite de nós atingido! Interrompendo execução.")

        return None, nodes_expanded, time.time() - start_time

    if current_node.state == GOAL_STATE:

        return get_path(current_node), nodes_expanded,
time.time() - start_time

    if current_node.state not in visited:

        visited.add(current_node.state)

        for child in get_successors(current_node):

            stack.append(child)

return None, nodes_expanded, time.time() - start_time

```

2.2 Busca em Profundidade Limitada (DLS)

A DLS é uma variação da DFS com um limite máximo de profundidade. Evita buscas infinitas, mas depende da escolha do limite e com isso não garante solução ótima. É mais controlada que a DFS, porém pode falhar se o limite for baixo.

```

def dls(initial_state, limit):

```

```

start_time = time.time()

start_node = Node(initial_state, depth=0)

stack = [start_node]
visited = set()
nodes_expanded = 0

while stack:
    current_node = stack.pop()
    nodes_expanded += 1

    if current_node.state == GOAL_STATE:
        return get_path(current_node), nodes_expanded,
time.time() - start_time

    if current_node.depth < limit:
        if current_node.state not in visited:
            visited.add(current_node.state)
            for child in get_successors(current_node):
                stack.append(child)

return None, nodes_expanded, time.time() - start_time

```

2.3 Busca em Aprofundamento Iterativo (IDS)

A IDS executa várias DFS limitadas, aumentando gradualmente o limite. Garantindo uma solução ótima com baixo uso de memória e reexplora estados. Combinando as vantagens da DFS e BFS.

```
def dls(initial_state, limit):  
    start_node = Node(initial_state, depth=0)  
  
    stack = [start_node]  
    visited = set()  
    nodes_expanded = 0  
  
    while stack:  
        current_node = stack.pop()  
        nodes_expanded += 1  
  
        if current_node.state == GOAL_STATE:  
            return get_path(current_node), nodes_expanded  
  
        if current_node.depth < limit:  
            if current_node.state not in visited:  
                visited.add(current_node.state)  
  
                for child in get_successors(current_node):  
                    stack.append(child)  
  
    return None, nodes_expanded  
  
def ids(initial_state, max_depth=30):  
    start_time = time.time()  
    total_nodes_expanded = 0  
  
    # Vai rodar a DLS aumentando o limite de 0 até max_depth
```

```

    for depth in range(max_depth):
        result, nodes_in_this_depth = dls(initial_state,
limit=depth)

        total_nodes_expanded += nodes_in_this_depth

        # Se encontrou a solução (result não é None), encerra o
algoritmo

        if result is not None:
            return result, total_nodes_expanded, time.time() -
start_time

    return None, total_nodes_expanded, time.time() - start_time

```

3.1 Algoritmo A* (Busca Informada)

O A* utiliza heurísticas para guiar a busca: $f(n)=g(n)+h(n)$

Com isso garante solução ótima com heurística admissível é muito eficiente, mas depende da qualidade da heurística. Foram utilizadas duas heurísticas: Peças fora do lugar e Distância de Manhattan.

```

# Heurística 1: Peças fora do lugar
def h_misplaced(state):
    return sum(1 for i in range(9) if state[i] != 0 and state[i]
!= GOAL_STATE[i])

# Heurística 2: Distância de Manhattan
def h_manhattan(state):
    distance = 0

```

```

    for i in range(9):
        if state[i] != 0:
            goal_pos = GOAL_STATE.index(state[i])
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(goal_pos, 3)
            distance += abs(x1 - x2) + abs(y1 - y2)

    return distance

def astar(initial_state, heuristic):
    start_time = time.time()

    # O nó inicial tem custo g=0 e h calculado
    start_node = Node(initial_state, g=0,
h=heuristic(initial_state))

    open_list = []
    heapq.heappush(open_list, start_node)

    visited = {}
    nodes_expanded = 0

    while open_list:
        current_node = heapq.heappop(open_list)
        nodes_expanded += 1

        if current_node.state == GOAL_STATE:
            return get_path(current_node), nodes_expanded,
time.time() - start_time

```



```
        # Se já visitamos este estado com um custo menor ou
igual, pulamos

        if current_node.state in visited and
visited[current_node.state] <= current_node.g:

            continue

        visited[current_node.state] = current_node.g

        for child in get_successors(current_node, heuristic):

            heapq.heappush(open_list, child)

    return None, nodes_expanded, time.time() - start_time
```

A heurística de peças fora do lugar consiste em contar quantas peças do tabuleiro não estão na posição correta em relação ao estado objetivo, comparando o estado atual com o estado final. Quanto mais peças fora do lugar, mais distante o estado está da solução. Essa heurística não considera a distância real das peças. Contudo, uma peça pode estar muito longe da posição correta, mas ainda assim conta apenas como 1.

A heurística de distância de Manhattan calcula quantos movimentos cada peça precisa fazer para chegar até sua posição correta. Para cada peça calcula a distância entre a posição atual e a posição correta, somando os movimentos na vertical e na horizontal. Portanto, o valor total é a soma das distâncias de todas as peças

Comparação dos Resultados

Para a realização dos testes, usamos dois exemplos de estados iniciais válidos de tabuleiro, organizando os resultados obtidos em duas planilhas que se encontram abaixo:

- Teste 1: tabuleiro = (1,2,3,5,0,6,4,7,8)

	Busca em Largura (BFS)	Busca em Profundidade (DFS)	Busca em Profundidade Limitada (DLS) limit = 20	Busca em Aprofundamento Iterativo (IDS)	Algoritmo A* (Heurística 1)	Algoritmo A* (Heurística 2)
Quantidade de movimentos	20	26	4	4	4	4
Nós explorados	39278	40	14117	76	5	5
Tempo de execução (segundos)	0.2837	0.000115	0.023234	0.000107	0.000087	0.000090

- Teste 2: tabuleiro = (8,0,6,5,4,7,2,3,1)

	Busca em Largura (BFS)	Busca em Profundidade (DFS)	Busca em Profundidade Limitada (DLS) limit = 20	Busca em Aprofundamento Iterativo (IDS) max_depth= 40	Algoritmo A* (Heurística 1)	Algoritmo A* (Heurística 2)
Quantidade de movimentos	27	19019	Não encontrou	37	27	27
Nós explorados	165017	30483	22227	1787172	101591	6715
Tempo de execução (segundos)	0.8470	0.118983	0.034501	3.165903	1.237530	0.120738

Conclusão

A análise dos testes evidencia as relações entre otimalidade, tempo e uso de memória de cada algoritmo avaliado. As buscas não informadas (todas menos A*) mostraram limitações claras em cenários complexos (como no caso do Teste 2):

- BFS garantiu a otimalidade (27 movimentos), mas à custa de uma expansão massiva de estados (165.017 nós), demandando alta memória;
- DFS encontrou uma solução de forma rápida, porém altamente subótima (19.019 movimentos);
- DLS falhou ao atingir seu limite de 20 níveis;
- IDS, embora não sofra com limites estritos, penalizou o tempo de execução ao reexplorar exaustivamente mais de 1,7 milhão de nós.

O grande destaque do estudo foi o algoritmo A*, que manteve a garantia de solução ótima resolvendo a ineficiência das buscas cegas. A comparação no Teste 2 provou que a qualidade da heurística é o fator mais determinante para o desempenho.

Consideração Final

O estudo do quebra cabeças deslizante demonstrou, na prática, a importância de selecionar a técnica de Inteligência Artificial mais adequada conforme os recursos computacionais disponíveis e os requisitos do problema. Este trabalho consolida o entendimento de que, à medida que a complexidade do problema cresce, o uso de buscas informadas se torna imprescindível. O uso do algoritmo A* aliado a uma função heurística adequada mostrou ser a abordagem definitiva para conciliar baixo custo de memória, rapidez e a obtenção do melhor caminho possível.